

ICT171 Cloud Server Project

Assignment 3 - 2026 Semester 1

Student Name: Sayeed Bin Akhter

Student Number: 35820304

GitHub Repository: <https://github.com/sayeed123526/ICT171-Cloud-Server-Project>

Video Explainer: <https://www.youtube.com/watch?v=3YaRDjMKqfk>

Global IP Address: 20.70.130.74

DNS Entry / Domain: <https://projectofict171.com>

Overview

This document describes how I provisioned, configured, and deployed a public-facing web server in the cloud for ICT171. The server runs a Cybersecurity Awareness website hosted at <https://projectofict171.com>. I used Microsoft Azure as an Infrastructure as a Service (IaaS) platform, meaning I had full SSH access to the machine and manually configured every component from the operating system up. The goal of this document is to be detailed enough that another ICT171 student or I could rebuild this server from scratch without doing any additional research.

Infrastructure

Component	Detail
Cloud Provider	Microsoft Azure (Student Subscription)
VM Name	ict171Sayeed
Network Interface	ict171sayeed499_z2
Tier	Standard – Regional
Public IP Address	20.70.130.74

Component	Detail
Internal IP	10.0.0.4 (eth0)
Operating System	Ubuntu 24.04.4 LTS
Kernel	6.17.0-1008-azure x86_64
Web Server	Apache2
Domain	projectofict171.com
SSL Certificate	Let's Encrypt (Certbot)
Version Control	GitHub — sayeed123526

Provisioning the Virtual Machine

I created the VM through the Azure portal at <https://portal.azure.com> using my Murdoch University student account, which provides free Azure credits.

Steps taken in the portal:

1. Click Create a resource → search Ubuntu Server 24.04 LTS → Create
2. Create a new Resource Group
3. Set VM name to ict171Sayeed, region to Australia East
4. Under Administrator account, select SSH public key, set username to azureuser
5. Generate a new key pair - Azure prompts you to download the .pem file. Save it as ict171azurekey.pem
6. Under Inbound port rules, allow SSH (22)
7. Click Review + Create → Create

After deployment, navigate to Networking → Add inbound port rule and add rules for:

Port	Protocol	Name
80	TCP	HTTP
443	TCP	HTTPS
22	TCP	SSH (already present)

These rules are set in the Network Security Group ict171Sayeed-nsg, attached to interface ict171sayeed499_z2.

Connecting via SSH

The .pem key file is stored locally at C:\Users\User\Documents\ict159\ict1592\ict171azurekey.pem. Connection is made from Windows PowerShell - noting that sudo is a Linux command and will not work in PowerShell itself; it only operates once inside the Ubuntu SSH session.

Connect from Windows PowerShell:

```
ssh azureuser@20.70.130.74 -i "C:\Users\User\Documents\ict159\ict1592\ict171azurekey.pem"
```

A successful connection shows the Ubuntu 24.04.4 LTS welcome banner with system load, memory usage, and kernel version. The internal prompt changes to:

```
azureuser@ict171Sayeed:~$
```

All sudo commands are run from this point on.

Initial System Update

On first login, the system indicated a restart was required. I rebooted cleanly:

```
sudo reboot
```

After reconnecting via SSH, I updated the package list and upgraded all installed packages:

```
sudo apt update
```

```
sudo apt upgrade -y
```

Installing Apache2

Apache2 is the web server that receives HTTP requests and serves the website files.

```
sudo apt install apache2 -y
```

Start Apache and enable it to launch automatically after any future reboot:

```
sudo systemctl start apache2
```

```
sudo systemctl enable apache2
```

Verify it is running:

```
sudo systemctl status apache2
```

The output should include Active: active (running).

Configure UFW firewall on the server:

```
sudo ufw allow OpenSSH
```

```
sudo ufw allow "Apache Full"
```

```
sudo ufw enable
```

```
sudo ufw status
```

Apache Full opens both port 80 and port 443. OpenSSH keeps port 22 open, preventing loss of SSH access after enabling UFW.

Deploying the Website

The website files (index.html and style.css) were transferred to the server.

Because scp must be run from the local Windows machine (not from inside the SSH session), I opened a separate PowerShell window for the upload:

```
scp -i "C:\Users\User\Documents\ict 159\ict1592\ict171azurekey.pem" index.html  
azureuser@20.70.130.74:/tmp/
```

```
scp -i "C:\Users\User\Documents\ict 159\ict1592\ict171azurekey.pem" style.css  
azureuser@20.70.130.74:/tmp/
```

Then, back in the SSH session, I moved the files into the Apache web root and set ownership so Apache can read them:

```
sudo mv /tmp/index.html /var/www/html/index.html
```

```
sudo mv /tmp/style.css /var/www/html/style.css
```

```
sudo chown www-data:www-data /var/www/html/index.html /var/www/html/style.css
```

```
sudo chmod 644 /var/www/html/index.html /var/www/html/style.css
```

Verify both files are present:

```
ls -la /var/www/html/
```

Configuring the Domain Name

The domain projectofict171.com was registered through a domain registrar. Two DNS A records were created pointing to the Azure VM's public IP:

Type	Host	Value
A	@	20.70.130.74
A	www	20.70.130.74

Apache VirtualHost configuration:

```
sudo nano /etc/apache2/sites-available/projectofict171.com.conf
```

After:

```
<VirtualHost *:80>
```

```
    ServerAdmin admin@projectofict171.com
```

```
    ServerName projectofict171.com
```

```
    ServerAlias www.projectofict171.com
```

```
    DocumentRoot /var/www/html
```

```
    ErrorLog ${APACHE_LOG_DIR}/error.log
```

```
    CustomLog ${APACHE_LOG_DIR}/access.log combined
```

```
</VirtualHost>
```

Saved with Ctrl+O, Enter, Ctrl+X. Then enabled the new site and disable the default:

```
sudo a2ensite projectofict171.com.conf
```

```
sudo a2disssite 000-default.conf
```

```
sudo systemctl reload apache2
```

Enabling HTTPS with Let's Encrypt

HTTPS encrypts all traffic between the user's browser and the server. I used Certbot, which automates the process of obtaining and installing a free SSL certificate from Let's Encrypt. This was done manually via the command line to satisfy the IaaS requirement of the assignment.

```
sudo apt install certbot python3-certbot-apache -y
```

```
sudo certbot --apache -d projectofict171.com -d www.projectofict171.com
```

Certbot prompted for an email address, asked to agree to the terms of service, and then automatically modified the Apache configuration to redirect all HTTP traffic to HTTPS. Certificates are valid for 90 days and auto-renew. I tested renewal:

```
sudo certbot renew --dry-run
```

The output confirmed Congratulations, all simulated renewals succeeded.

Bash Script - Server Health Check

The assignment requires a commented script. Rather than a simple status check, I wrote a monitoring script that inspects five aspects of server health and automatically recovers the web server if it has stopped. It checks: (1) the Apache service, restarting it if it is down (2) the website's HTTP response code (3) disk usage (4) memory usage and (5) the number of days remaining on the SSL certificate. Every run is written to `/var/log/healthcheck.log` with a timestamp and an overall status of OK, WARN, RECOVERED, or CRITICAL, and the terminal output is colour-coded for quick reading.

My contribution beyond the lab material is the auto-restart logic, the disk/memory/certificate-expiry checks, the single-line structured log format, and the colour-coded status - turning a basic check into a practical monitoring tool that can be scheduled with cron. The full commented script is in the repository as `healthcheck.sh`.

To run it:

```
chmod +x healthcheck.sh
```

```
sudo ./healthcheck.sh
```

To run it automatically every 5 minutes, (with `sudo crontab -e`) :

```
* /5 * * * * /home/azureuser/healthcheck.sh >/dev/null 2>&1
```

Verifiable output –

```
azureuser@ict171sayeed: $ chmod +x ~/healthcheck.sh
azureuser@ict171sayeed: $ sudo ~/healthcheck.sh
k.sh
cat /var/log/healthcheck.logsudo ~/healthcheck.sh
sudo ~/healthcheck.sh
cat /var/log/healthcheck.log 2024-06-01 14:01:22 [OK] Apache2: RUNNING | Website: ONLINE (HTTP 200) | Disk: 100% | Memory: 40% | SSL: 41 days left
azureuser@ict171sayeed: $
```

The output can be independently verified by visiting <https://projectofict171.com> in a browser - a successful page load confirms an HTTP 200 response.

GitHub Repository

All project files are version-controlled at <https://github.com/sayeed123526/ICT171-Cloud-Server-Project>.

Repository structure:

ICT171-Cloud-Server-Project/

├── README.md

├── index.html

├── style.css

├── healthcheck.sh

└── docs/

 └── report.pdf

Pushing files to GitHub from the server:

```
cd ~
```

```
git clone https://github.com/sayeed123526/ICT171-Cloud-Server-Project.git
```

```
cp /var/www/html/index.html ~/ICT171-Cloud-Server-Project/
```

```
cp /var/www/html/style.css ~/ICT171-Cloud-Server-Project/
```

```
cp ~/healthcheck.sh ~/ICT171-Cloud-Server-Project/
```

```
cd ICT171-Cloud-Server-Project
```

```
git add .
```

```
git commit -m "Add website files, stylesheet and health check script"
```

git push origin main

Cost

Resource	Cost
Azure VM (Standard B1s)	\$0 - Azure Student subscription
Public IP address	\$0 - included with student VM
Domain name (projectofict171.com)	~\$12 AUD/year
SSL Certificate (Let's Encrypt)	\$0

The Azure Student subscription provides \$100 USD in free credits, sufficient to run a B1s VM continuously for several months.

Reflection

This project required provisioning a virtual machine from scratch, resolving a port conflict caused by a pre-installed nginx service, understanding the separation between Windows PowerShell (used for SSH and SCP) and the Ubuntu shell (where all server commands run), and configuring DNS, Apache virtual hosts, and HTTPS manually. The most instructive challenge was diagnosing why the website was reachable via curl http://localhost on the server but not from a browser - ultimately traced to the Azure public IP not forwarding traffic, which was resolved by verifying the IP association in the Azure portal. These are realistic problems that any junior sysadmin would encounter and the process of diagnosing and fixing them has been as valuable as the final working result.

References

Microsoft, "Use SSH keys to connect to Linux VMs," *Microsoft Learn*, Oct. 2024. [Online]. Available: <https://learn.microsoft.com/en-us/azure/virtual-machines/linux/ssh-from-windows>

Canonical Ltd., "Install and Configure Apache," *Ubuntu Tutorials*, 2024. [Online]. Available: <https://ubuntu.com/tutorials/install-and-configure-apache>

DigitalOcean, "How To Set Up Apache Virtual Hosts on Ubuntu," *DigitalOcean Community*, Apr. 2022. [Online]. Available: <https://www.digitalocean.com/community/tutorials/how-to-set-up-apache-virtual-hosts-on-ubuntu-20-04>

Electronic Frontier Foundation, "Certbot Instructions: Apache on Ubuntu," *Certbot*, 2024. [Online]. Available: <https://certbot.eff.org/>

Microsoft, "Associate a public IP address to a virtual machine," *Microsoft Learn*, Nov. 2025. [Online]. Available: <https://learn.microsoft.com/en-us/azure/virtual-network/ip-services/associate-public-ip-address-vm>

Murdoch University Library, "Referencing," 2024. [Online]. Available: <http://library.murdoch.edu.au/Students/Referencing/>

Murdoch University, "Academic Integrity," 2024. [Online]. Available: <http://our.murdoch.edu.au/Student-life/Study-successfully/Study-Skills/Referencing/Academic-integrity/>